

Atividade Prática

Refatoração de Código Legado com Clean Code

Java | Individual | 10,0 pontos | Entrega no Moodle

1. Contexto

Você recebeu a tarefa de assumir a manutenção de um sistema de gestão de biblioteca universitária desenvolvido há alguns anos por um programador que não seguia boas práticas de desenvolvimento. O código funciona, mas qualquer modificação leva horas por causa da dificuldade de leitura e da falta de estrutura.

O projeto é composto por quatro classes Java localizadas no pacote `br.edu.puc.biblioteca` do projeto `biblioteca-legado` (Maven). Seu objetivo é identificar as violações de Clean Code presentes no código e refatorar o projeto, produzindo uma versão mais legível, testável e manutenível.

O comportamento funcional deve ser preservado. Refatoração, por definição, altera a estrutura interna sem modificar o comportamento externo. Testes unitários são a rede de segurança que garante isso.

2. Estrutura do Projeto Legado

O projeto fornecido contém as seguintes classes:

```
biblioteca-legado/  
src/main/java/br/edu/puc/biblioteca/  
  Usr.java      <- representa um usuario do sistema  
  Livro.java    <- representa um livro do acervo  
  Bib.java      <- gerencia todas as operacoes (emprestimo, cadastro, busca)  
  Main.java     <- demonstracao de uso  
pom.xml
```

O sistema suporta: cadastrar usuarios (alunos e professores), cadastrar livros, emprestar livros, devolver livros e buscar livros por titulo ou autor. Alunos podem ter ate 3 livros emprestados simultaneamente; professores, ate 5.

3. Parte 1 — Identificacao de Violacoes (3,0 pts)

Leia o codigo do projeto com atencao e elabore um documento listando as violacoes de Clean Code encontradas. Para cada violacao identificada, informe:

- o arquivo e o elemento afetado (classe, metodo, campo ou variavel),
- o principio de Clean Code violado (nomenclatura, funcoes, comentarios, formatacao, objetos e estruturas, tratamento de erros, classes),
- uma descricao breve do problema.

Criterio	Pontuacao	Observacoes
Violacoes de nomenclatura (classes, metodos, campos e variaveis)	1,0 pt	Minimo 5 violacoes identificadas
Violacoes de funcoes (tamanho, responsabilidade, parametros)	1,0 pt	Minimo 3 violacoes identificadas
Violacoes de erros e classes (null, SRP, God Class)	1,0 pt	Minimo 2 violacoes por subcategoria

4. Parte 2 — Refatoracao (5,0 pts)

Crie um novo projeto Maven (biblioteca-refatorado) e refatore o codigo aplicando os principios de Clean Code estudados. A refatoracao deve contemplar obrigatoriamente os dez itens a seguir. Cada item vale 0,5 ponto.

Item 1 — Nomenclatura de classes, campos e metodos

Renomeie todas as classes, campos e metodos que violam as convencoes de nomenclatura Java e os principios de Clean Code. Os nomes devem revelar intencao sem necessidade de comentario. Exemplos: `Usr` para `Usuario`, `nm` para `nome`, `bsc()` para `buscarLivros()`, `chkAtr()` para `estaAtrasado()`.

Item 2 — Encapsulamento

Torne todos os campos privados. Exponha apenas o que for necessario por meio de metodos com nomes expressivos. A decisao sobre o que expor deve ser guiada pelo comportamento, nao pela estrutura interna.

Item 3 — Enum no lugar de constante inteira

O campo `tp` de `Usr` usa inteiros (1 = aluno, 2 = professor) e os limites de emprestimo sao verificados por condicionais espalhados. Crie um enum `TipoUsuario` com os valores `ALUNO` e `PROFESSOR`, cada um encapsulando seu proprio limite de emprestimos.

Item 4 — LocalDate no lugar de String para data

O campo `dt` de `Livro` armazena a data como `String`, o que impossibilita calculos de prazo. Substitua por `java.time.LocalDate` e implemente corretamente o calculo de dias em atraso, eliminando o `TODO` presente no metodo `chkAtr()`.

Item 5 — Decompor Bib.proc() em metodos focados

O metodo `Bib.proc()` executa quatro operacoes completamente diferentes, selecionadas por um parametro inteiro. Separe-o em metodos com nomes claros, um para cada operacao: `emprestarLivro()`, `devolverLivro()`, `cadastrarLivro()` e `cadastrarUsuario()`. Cada metodo deve fazer uma coisa.

Item 6 — Eliminar parametros booleanos

Os metodos `proc()` e `lst()` recebem flags booleanas que mudam fundamentalmente o comportamento. Elimine esses parametros dividindo os metodos ou extraindo o comportamento condicional em metodos separados com nomes que revelam a intencao.

Item 7 — Substituir numeros magicos por constantes nomeadas

O codigo original contem os valores 1, 2, 3, 5 e 2.5 sem qualquer explicacao de significado. Substitua todos por constantes com nomes em `UPPER_SNAKE_CASE`: `LIMITE_EMPRESTIMOS_ALUNO`, `LIMITE_EMPRESTIMOS_PROFESSOR`, `MULTA_POR_DIA_DE_ATRASO`, `PRAZO_EMPRESTIMO_EM_DIAS`.

Item 8 — Substituir retorno null por Optional ou excecao

O metodo `bsc()` retorna null quando nao ha resultado, e `proc()` retorna null para indicar falha. Substitua retornos null por `Optional<T>` em buscas e por excecoes especificas (`LivroNaoEncontradoException`, `LivroIndisponivelException`, `LimiteEmprestimosException`) em operacoes que podem falhar.

Item 9 — Eliminar codigo duplicado (DRY)

O loop de busca por ISBN e o loop de busca por matricula aparecem repetidos em varios pontos de `Bib.proc()`. Extraia cada busca em um metodo privado reutilizavel, eliminando a duplicacao.

Item 10 — Aplicar SRP: separar responsabilidades de Bib

A classe `Bib` e uma God Class: gerencia o catalogo, os usuarios, os emprestimos e a apresentacao. Separe as responsabilidades em pelo menos duas classes distintas. Sugestao: `CatalogoService` (cadastro e busca) e `EmprestimoService` (emprestimo e devolucao).

5. Parte 3 — Testes Unitarios (2,0 pts)

Escreva exatamente quatro testes unitarios com JUnit 5, cobrindo os cenarios abaixo. Cada teste vale 0,5 ponto.

1. Emprestimo bem-sucedido: livro disponivel e usuario dentro do limite.
2. Emprestimo falha quando o livro ja esta emprestado.
3. Emprestimo falha quando o usuario atingiu o limite de emprestimos do seu tipo.
4. Devolucao libera o livro e decrementa o contador de emprestimos do usuario.

Os testes devem ser independentes entre si e nao depender de banco de dados, rede ou qualquer estado externo. Use `@BeforeEach` para o setup comum.

6. Criterios de Entrega

- Arquivo ZIP contendo o projeto Maven refatorado (biblioteca-refatorado).
- Documento PDF ou DOCX com a lista de violacoes identificadas na Parte 1.
- O projeto refatorado deve compilar sem erros (`mvn compile`).

- Os testes devem passar (mvn test).
- Não altere os arquivos do projeto-legado; entregue apenas o projeto refatorado.

Prazo e forma de entrega: conforme informado no Moodle. Atividades entregues após o prazo receberão desconto de 1,0 ponto por dia de atraso.

7. Rubrica Geral

Criterio	Pontuacao	Observacoes
Parte 1: Identificacao de violacoes	3,0 pts	1,0 pt por categoria
Parte 2: Refatoracao (10 itens)	5,0 pts	0,5 pt por item concluido
Parte 3: Testes unitarios (4 testes)	2,0 pts	0,5 pt por teste correto
TOTAL	10,0 pts	

Critérios de desclassificação parcial: (a) código que não compila zera a Parte 2 independentemente das outras partes; (b) código idêntico ao de outro aluno recebe nota zero em ambas as entregas.